

```

    }
    }
    public class TestMyThread
    {
    public static void main( String[] args )
    {
    MyThread pat = new MyThread();
    pat.start();
    Thread.dumpStack();
    }
    }

```

We start off with our typical class `MyThread` that extends `Thread`. We then instantiate a thread object, and execute the `start()` method. Now, because the thread method `dumpStack()` is static, we call it on the class object.

```

MyThread Constructor
java.lang.Exception: Stack trace
java.lang.Throwable(java.lang.String)
java.lang.Exception(java.lang.String)
void java.lang.Thread.dumpStack()
void TestMyThread.main(java.lang.String [])
After dumpStack()

```

You see `main()` was the first thing executed. You read a stack dump from the bottom up, the way it was stacked. Next, `Thread`'s static method `dumpStack()` was called. Finally `Exception` was called, followed by `Throwable`, the Superclass.

8.4.5 final void setPriority(int p)

This method allows you the change the priority to an integer between 1 and 10.

8.4.6 final int getPriority()

This method lets you return the priority for a thread. Notice, this method is final to prevent you from causing any mischief.

8.5 Thread states

Life Cycle of a Thread:

- born - Thread is Instantiated. When `start()` method is called, thread enters ready state.